National University of Sciences and Technology (NUST)

SE321: Software Quality Engineering

Project Idea Proposal

Test Plan for:

REACH

(Realtime Emergency Alert Collection Hub)

Submitted By:

Danish Munib
461050
BESE-14 B

Rafay Ahmad
478383
BESE-14 B

Ahmad Shahmeer
475027
BESE-14 B

Instructor:
Ayesha Kanwal
School of Electrical Engineering & Computer Science

April 14, 2026

1. Project Summary

Pakistan faces increasing climate disasters like floods and wildfires, but its disaster warning system is fragmented and inefficient. Critical alerts from different agencies are buried in lengthy PDFs or scattered across social media, leading to dangerous delays and a failure to deliver clear, actionable warnings to the communities that need them most. This information gap was starkly highlighted by the 2025 floods, which resulted in over 1,000 fatalities and affected 6.9 million people.

REACH (Real-time Emergency Alert Collection Hub) is an AI-powered system designed to bridge this gap. It automatically collects disaster alerts from official sources like the NDMA and PMD, processes them using artificial intelligence, and delivers precise, location-targeted warnings directly to citizens' mobile devices and web browsers in near-real-time.

Implementation:

- **Core Technology:** The system uses a three-tier architecture:
    1. **Collection:** Automated "scrapers" gather data from official websites, PDFs, and APIs.
    2. **Processing:** An AI engine uses Natural Language Processing (NLP) and Vision-Language Models (VLMs) to extract key details (location, severity, type) from text and visual maps, transforming technical jargon into clear, actionable alerts.
    3. **Distribution:** A multi-channel notification system delivers these tailored alerts via mobile apps, push notifications, and SMS.
- **Technical Innovation:** For the project, we used "compositional geocoding" to interpret vague location descriptions from these reports (e.g., "areas downstream of Tarbela Dam") and translate them into precise geographical boundaries for targeted warnings, avoiding generic province-wide alerts.
- **Development:** The project was implemented in phases, starting with building the core infrastructure and data processing capabilities, then integrating notification systems and community feedback features, before a final handoff for full-scale operation.

In essence, REACH acts as a critical link, ensuring that vital disaster information reaches the right people, in the right place, in a format they can understand and act upon, ultimately saving lives and building community resilience.

Key features and modules:

- Continuous Scraper: Polls official government sources (NDMA, PMD) every 10 minutes for new reports
- AI Interpretation Engine: Uses Gemini Flash in reasoning mode with few-shot prompting to parse text, maps, and charts from documents into CAP-compliant JSON
- Custom Geocoder: PostGIS-backed geocoder that resolves compositional region descriptions (e.g., "areas downstream of Tarbela Dam") to district polygons
- PostgreSQL Data Store: Denormalized read model with spatial indexes for high-performance queries
- Interactive Map Dashboard: React.js + Mapbox frontend allowing users to filter by disaster type, search by location, and view distilled alert summaries
- Notification Infrastructure (Planned): Geo-fenced push notifications and mobile app support

Technology Stack:

Layer	Technology	Purpose
AI / LLM	Gemini Flash (Google AI Studio)	Document interpretation, geocoding heuristics
Scraping	BeautifulSoup, Python	Data extraction from government portals
Database	PostgreSQL + PostGIS	Structured storage with spatial support
Backend API	FastAPI (Python)	REST API serving alert data
Frontend	React.js + Mapbox GL JS	Interactive alert dashboard
Infrastructure	Supabase, Render, Modal	Managed DB, API hosting, async workers

Scope of Testing:

The testing scope covers three primary layers of the system:

- **Data Pipeline (Unit + Integration):** Scraper correctness, JSON schema validation of AI output, geocoding accuracy, database write integrity
- **Backend API (Integration):** All REST endpoints with correctness, error handling, authentication, and data integrity
- **Frontend + End-to-End (System):** Map rendering, alert filtering, search functionality, performance under load, and basic security posture

The third-party Gemini API internals, Mapbox rendering engine internals, and Supabase infrastructure are out of scope for this engagement.

---

2. Testing Overview

The REACH platform falls under the **Web Development** category. Our quality assurance strategy is designed to rigorously validate both the FastAPI backend (including the AI processing pipeline and PostGIS custom geocoding) and the React.js frontend.

The testing life cycle is divided into three distinct levels:

1. **Unit Testing:** Ensuring the correctness of discrete functions, components, and geocoding heuristics in isolation using mocked dependencies.
2. **Integration Testing:** Validating API contracts, database read/writes, and interactions between the frontend, backend, and external LLM/Geocoding services.
3. **System-Level Testing:** Simulating real-world end-to-end scenarios, including functional GUI testing, load testing for concurrency during high-alert periods, and basic security testing.

Testing Tools:

To achieve comprehensive coverage, we have selected the following advanced testing tools:

1. **PyTest (Backend Unit & Integration):** Used for unit testing Python microservices, FastAPI routes, and the custom geocoding logic.
2. **Jest & React Testing Library (Frontend Unit):** Used to test React UI components, state management (hooks), and utility functions.

3. **Postman & Newman (API Integration):** Used for developing automated API test suites to validate REST contracts and expected HTTP responses.
4. **Cypress (System/E2E Testing):** Used for GUI-based automated functional testing of user workflows on the web dashboard.
5. **k6 (Performance Testing):** Used to simulate concurrent users accessing the dashboard and APIs during a hypothetical disaster event to ensure system stability.

---

3. Test Plans

3.1. Unit Testing:

Requirements:

- **Coverage Goal:** Minimum 80% code coverage across frontend and backend modules.
- **Workload:** 30 unit test cases distributed evenly (10 per group member).
- **Execution:** External dependencies (e.g., Supabase, Gemini Flash, Mapbox, Redis) will be mocked/stubbed to isolate the specific logic under test.

Tools:

- **Backend:** PyTest, pytest-cov (Coverage), unittest.mock
- **Frontend:** Jest, React Testing Library, Istanbul (Coverage)

Test Cases:

Backend Geocoding (10 cases)

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M1-001	DirectionalParser	parse	Parse simple directional phrase	Parser initialized	"Central Sindh"	Direction = CENTRAL, place tokens include "Sindh"	To be filled

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M1-002	DirectionalParser	parse	Parse input with no direction	Parser initialized	"Islamabad"	Direction = None, place tokens include "Islamabad"	
UT-M1-003	DirectionalParser	parse	Handle empty/white space safely	Parser initialized	" "	No exception, empty-safe output	
UT-M1-004	DirectionalParser	parse	Case/hyphen-insensitive directional parsing	Parser initialized	"north-eastern KPK", "NORTHEASTERN KPK"	Same directional result for variants	
UT-M1-005	NameMatcher	_select_best_candidate	Prefer highest similarity when gap is significant	Candidate list mocked	Candidates with clear score leader	Candidate with top similarity selected	
UT-M1-006	NameMatcher	_select_best_candidate	Prefer lower admin level when scores are within tolerance	Candidate list mocked	Similar scores, diff hierarchy levels	More specific admin level selected	
UT-M1-007	NameMatcher	match	Blank query returns no result	Repository mocked	""	Returns None, no fuzzy lookup call	
UT-M1-008	ExternalGeocoder	disambiguate_by_centroid	Pick coordinate nearest context centroid	Valid coords + context	3 candidates + 2 context points	Closest-to-centroid coordinate returned	
UT-M1-009	ExternalGeocoder	geocode	Repeated query uses cache	API + Redis mocked	Same location twice	First call hits API, 2nd call returns cached data	

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M1-010	GeocodingService	geocode_location	Fallback to external geocoding when fuzzy fails	Fuzzy matcher mocked to fail	Unknown place string	Non-crashing fallback path executed	

Backend Processing Engine (10 Cases)

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M2-001	doc_utils	to_base64	Convert PIL image to base64 JPEG	PIL image fixture available	Valid image object	Non-empty base64 string returned	To be filled
UT-M2-002	doc_utils	pdf_to_images	Convert single-page PDF to one image	PDF fixture available	Single-page PDF bytes	List length = 1	
UT-M2-003	doc_utils	pdf_to_images	Corrupted PDF handling	None	Corrupted PDF bytes	Controlled exception/propagated error	
UT-M2-004	doc_utils	fetch_file	Successful HTTP fetch returns bytes	HTTP client mocked	Valid URL with 200 response	Byte content returned	
UT-M2-005	doc_utils	url_to_b64_strings	Image URL conversion path	fetch_file mocked with image bytes	PNG/JPEG URL	One data URI in output list	
UT-M2-006	doc_utils	url_to_b64_strings	PDF URL conversion path	fetch_file + pdf_to_images mocked	PDF URL	One data URI per page	
UT-M2-007	LLMClient	__init__	Unsupported model key rejected	Config fixture loaded	Invalid model name	ValueError raised	

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M2-008	AsyncLLM Client	call	Runtime kwargs merged with defaults	API client mocked	Messages + custom kwargs	API called with merged params	
UT-M2-009	PipelineProcessor	_parse	Valid JSON parsed into domain objects	Parser dependencies mocked	Well-formed JSON string	Alert object + alert areas produced	
UT-M2-010	PipelineProcessor	_parse	Invalid enum/category rejected	Validation active	JSON with invalid category	Validation error path triggered	

Frontend Logic (10 Cases)

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M3-001	LRUCache	set/get	Retrieve inserted value and update recency	Cache initialized	set("a", v), get("a")	Returned value equals inserted value	To be filled
UT-M3-002	LRUCache	set	Evict least-recently-used when full	Max size small (e.g., 2)	Insert 3 keys w/ access pattern	LRU key evicted	
UT-M3-003	LRUCache	get	Expired entry returns undefined and is removed	TTL configured	Insert, wait TTL, get(key)	Undefined result, stale key removed	
UT-M3-004	alertsService	getAlertGeometry	Geometry cache used on repeated alertId	Supabase RPC mocked	Same alertId twice	RPC once, second result from cache	

Test ID	Module Name	Function Name	Test Description	Preconditions	Input / Test Data	Expected Output	Actual Output Pass/Fail Remarks
UT-M3-005	alertsService	invalidateStaleGeo	Remove cache entries not in active IDs	Cache pre-populated	Current IDs subset	Non-current keys removed	
UT-M3-006	useAlerts hook	initial fetch flow	autoFetch loads data and clears loading	alertsService mocked success	autoFetch = true	alerts populated, loading false	
UT-M3-007	useAlerts hook	error handling flow	Failed service call sets error and empties alerts	alertsService mocked failure	API error	error populated, alerts empty	
UT-M3-008	FilterPanel	severity mapping helper	Moderate default includes upward severities	Component logic isolated	default severity = Moderate	Returned severities match policy	
UT-M3-009	DateRangeSelector	date normalization	Start date after end date triggers swap	Component mounted	start > end selection	Callback receives corrected order	
UT-M3-010	RecentAlertsPanel	severity helper	Unknown severity uses fallback style	Helper callable	severity = "Unknown"	Default/fallback mapping returned	

3.2. Integration Testing:

Requirements:

- **Coverage Goal:** Minimum 15 test cases covering interactions between components (e.g., FastAPI + Supabase, Pydantic models + Routing).

- **Scope:** Verifying backend REST endpoints, payload validation, correct database read/write interactions, caching behavior, and error bridging.
- **Constraints:** Tests must use valid and invalid HTTP inputs and verify state changes in the underlying PostgreSQL database where applicable.

Tools:

- **Postman / Newman:** For declarative API endpoint validation and automated HTTP scenario sequences.
- **PyTest + HTTPX / FastAPI TestClient:** For programmatic integration inside the backend CI pipeline.

Test Cases (15 Total):

Test ID	Modules Involved	Test Scenario	API Endpoint	Request Data	Expected Response	Database Validation	Actual Result Pass/Fail Defect ID
IT-001	FastAPI Routes + Geocoding + PlacesRepo	Geocode a valid simple location	POST /api/v1/geocode	locations: ["Islamabad"]	200 OK, one result with matched_places not empty	Returned place_id exists in places table; no unintended writes	To be filled
IT-002	FastAPI Routes + DirectionalParser + Geocoding	Geocode directional region	POST /api/v1/geocode	locations: ["Central Sindh"]	200 OK, directional result with relevant places	All returned place_ids map to Sindh hierarchy in DB	
IT-003	FastAPI Routes + GeocodingService	Batch request with mixed valid locations	POST /api/v1/geocode	locations: ["Karachi","Lahore","Islamabad"]	200 OK, 3 results, errors empty	All matched place_ids resolve to valid records	
IT-004	FastAPI Routes + GeocodingService	Batch request with one unknown location	POST /api/v1/geocode	locations: ["Karachi","Xyz Unknown"]	200 OK, valid match for known city, error entry for unknown	Known city maps to DB record; unknown creates no DB mutation	
IT-005	FastAPI GET route + GeocodingService	Single location geocode via GET	GET /api/v1/geocode/Lahore	query: include_confidence=false	200 OK, one result object for Lahore	Returned place_id exists and hierarchy metadata is valid	

Test ID	Modules Involved	Test Scenario	API Endpoint	Request Data	Expected Response	Database Validation	Actual Result Pass/Fail Defect ID
IT-006	FastAPI Suggest route + NameMatcher	Typo suggestions retrieval	GET /api/v1/suggest/Islmabad	query: limit=5	200 OK, suggestions array includes Islamabad-like name	Suggested names exist in places table	
IT-007	FastAPI Health route	Health endpoint availability	GET /api/v1/health	none	200 OK, status=healthy, service/version present	No DB write/read required	
IT-008	FastAPI Validation + Pydantic Models	Validation: empty locations list rejected	POST /api/v1/geocode	locations: []	422 Unprocessable Entity with validation details	No DB read/write triggered	
IT-009	FastAPI Validation + Pydantic Models	Validation: malformed options type rejected	POST /api/v1/geocode	include_confidence...: "yes"	422 Unprocessable Entity	No DB read/write triggered	
IT-010	URL Path Handling + GeocodingService	GET geocode with encoded spaces	GET /api/v1/geocode/Northern%20Punjab	None	200 OK, parsed directional result returned	Result set aligned with Punjab geography records	
IT-011	FastAPI Routes + Geocoding + ExternalFallback	Unknown place triggers graceful failure	POST /api/v1/geocode	locations: ["Unknown@@@Area"]	200 OK with result error message, no crash	No invalid place inserted/updated in DB	
IT-012	FastAPI Error Handling + Dependency Injection	Internal service exception mapped to 500	POST /api/v1/geocode	valid body but service mocked to throw	500 Internal Server Error, generic detail message	No partial writes; transaction state unchanged	
IT-013	GeocodingService + RedisCache +	Cache integration on repeated geocode	POST /api/v1/geocode (twice)	locations: ["Islamabad"]	Both 200 OK; second response faster	Redis key exists after first call; second call serves cache	

Test ID	Modules Involved	Test Scenario	API Endpoint	Request Data	Expected Response	Database Validation	Actual Result Pass/Fail Defect ID
	ExternalFallback						
IT-014	API + DB + Service under concurrency	Concurrent requests stability	POST /api/v1/geocode	50 parallel requests, mixed locations	No 5xx, success rate >= 99%, consistent schema	No deadlocks; DB connections remain healthy	
IT-015	FastAPI Routes + NameMatcher	Suggest endpoint boundary for limit	GET /api/v1/suggest/Karacii	query: limit=1 and limit=10	200 OK, suggestions count respects limit and sorting	Suggested rows correspond to valid place records	

3.3. System Testing:

Requirements:

- **Coverage Goal:** Minimum 10 end-to-end scenarios executing standard user flows.
- **Scope:** Validate complete integrations from the frontend React UI down to the backend database. This includes verifying filtering mechanisms, Mapbox data layers, and performance/security parameters.
- **Automation:** Functional workflows must be automated using a GUI-based automation tool, simulating a real browser experience.

Tools:

- **Functional UI Automation:** Cypress (for clicking interfaces, verifying map rendering, and manipulating filters).
- **Performance/Load:** k6 (to simulate high traffic during hypothetical disaster escalation).
- **Security:** OWASP ZAP (Baseline scan against APIs and frontend routes to intercept XSS or baseline vulnerabilities).

Test Cases (10 Total, Functional + Non-Functional):

Test ID	Test Scenario	User Story / Req ID	Preconditions	Test Steps	Test Data	Expected Result	Environment	Actual Result Pass/Fail Severity
ST-001	Functional: User loads dashboard and map renders alerts	US-01 View live alerts	Frontend and backend running; sample alerts available	Open app URL, wait for map and alert list load	Default dataset	Map tiles render, alerts visible, no console errors	Win 11, Chrome latest	To be filled
ST-002	Functional: Filter by disaster type	US-02 Filter alerts by category	Alert data contains multiple disaster categories	Select Flood only, observe map/list	Category =Flood	Only Flood alerts remain in map and panels	Win 11, Chrome latest	
ST-003	Functional: Filter by severity	US-03 Prioritize severe alerts	Dataset has mixed severity levels	Apply Severe+Extreme filter	Severity values mixed	Display updates to selected severities only	Win 11, Firefox latest	
ST-004	Functional: Search by location	US-04 Find local alerts	Search enabled; known location exists	Enter location keyword, submit search	Lahore	Relevant area centers/highlights and list narrows	Win 11, Edge latest	
ST-005	Functional: Date range filter	US-05 View alerts by period	Historical alerts available	Set start/end dates; apply	Last 7 days	Results limited to chosen date window	Win 11, Chrome latest	
ST-006	Functional: View alert detail card/popup	US-06 Read concise alert summary	Alerts visible on map/list	Click alert marker/card	Any active alert	Detail panel shows title, severity, timing, source, location	Win 11, Chrome latest	
ST-007	Non-Functional (Performance): Initial load time target	NFR-PERF-01	Production-like build/network profile configured	Measure first meaningful render over 10 runs	Home dashboard	Median load <= 3s, p95 <= 5s	Win 11, Chrome, broadband	
ST-008	Non-Functional (Load): API and UI under	NFR-PERF-02	k6 scenario ready; monitored backend	Run 200 virtual users for 10 minutes	Mixed filter/search actions	Error rate < 1%, p95 API latency <= 2s, UI stays responsive	k6 + staging stack	

Test ID	Test Scenario	User Story / Req ID	Preconditions	Test Steps	Test Data	Expected Result	Environment	Actual Result Pass/Fail Severity
	concurrent users							
ST-009	Non-Functional (Security): Baseline vulnerability scan	NFR-SEC-01	Staging URL exposed for scan	Run OWASP ZAP baseline against frontend+API	Auth-less public routes	No High/Critical findings; Medium findings logged	OWASP ZAP baseline	
ST-010	Non-Functional (Compatibility): Cross-browser/mobile layout	NFR-UX-01	Supported browser matrix defined	Execute core flow in Chrome/Firefox + mobile viewport	1366x768 and 390x844	Layout usable, controls accessible, no blocking UI defects	Desktop + mobile emulation	